Advance Filter Module Documentation

Project: SmartOffice(HR Management System)
Module: Advance Filter Module
Version: 1.0
Prepared by: Sayed Al Momin
Date: 27 Jan 2026

1. Document Control

1.1 Version History

Version	Date	Description	Author
1.0	27-01-2026	Initial Documentation	Sayed Al Momin

2. Introduction

2.1 Purpose

This document describes the Advance Filter implementations of all modules list view with filter query according to model's attributes dynamically including's bellows operations….

EQ => '=',

NEQ => '!=',

CONTAINS => 'like',

NOT_CONTAINS => 'not like',

STARTS_WITH => 'like',

ENDS_WITH => 'like',

GT => '>',

LT => '<',

GTE => '>=',

LTE => '<=',

IN => 'in',

NOT_IN => 'not_in',

BETWEEN => 'between',

NOT_BETWEEN => 'not between',

2.2 Module Overview

The **Advanced Filter Module** is designed to provide a powerful, flexible, and scalable way to filter large datasets across the system. Instead of relying on fixed search fields, this module allows dynamic filtering based on multiple conditions, operators, and logical rules.It

3. Core Objectives

The Advanced Filter Module was built to:

- Support **dynamic query building**
- Allow **multiple filters at once**
- Handle **different data types** (string, number, date, boolean)
- Support **advanced operators** (EQ, NEQ, GT, LT, IN, BETWEEN, LIKE, etc.)
- Work seamlessly with **Laravel Eloquent & Query Builder**
- Provide a **standard API format** for frontend & mobile apps
- Improve **performance** and **code reusability**

Key Features

- Dynamic filter conditions from request payload
- Operator-based filtering system
- Nested AND / OR logic support
- Search by partial text, exact match, range, and sets
- Supports API, web panel, and mobile applications
- Central helper/service class (e.g., AdvancedFilterService)
- Secure and injection-safe query generation
- Easily extendable for new operators and fields

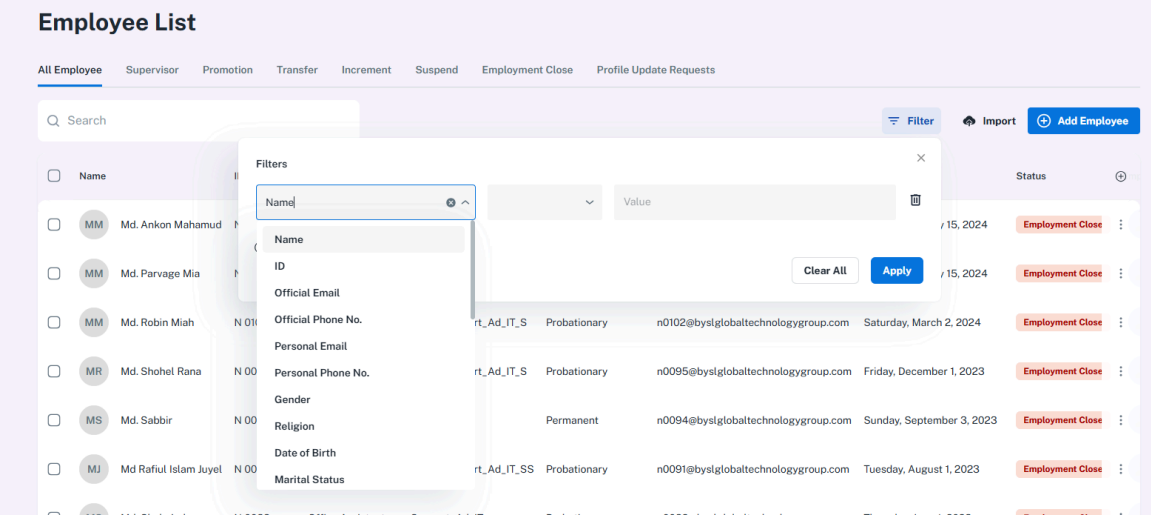
4. Where This Module Is Used

The **Advanced Filter** module allows users to dynamically search and filter employees based on **any attribute** stored in the system.
It supports **multiple conditions**, **different operators**, and **API-driven value suggestions** for fast and accurate filtering.

The Advanced Filter Module is commonly integrated into:

- Employee management listing
- Leave management
- Payroll reports

- Reports
- Any large data table with search & filtering

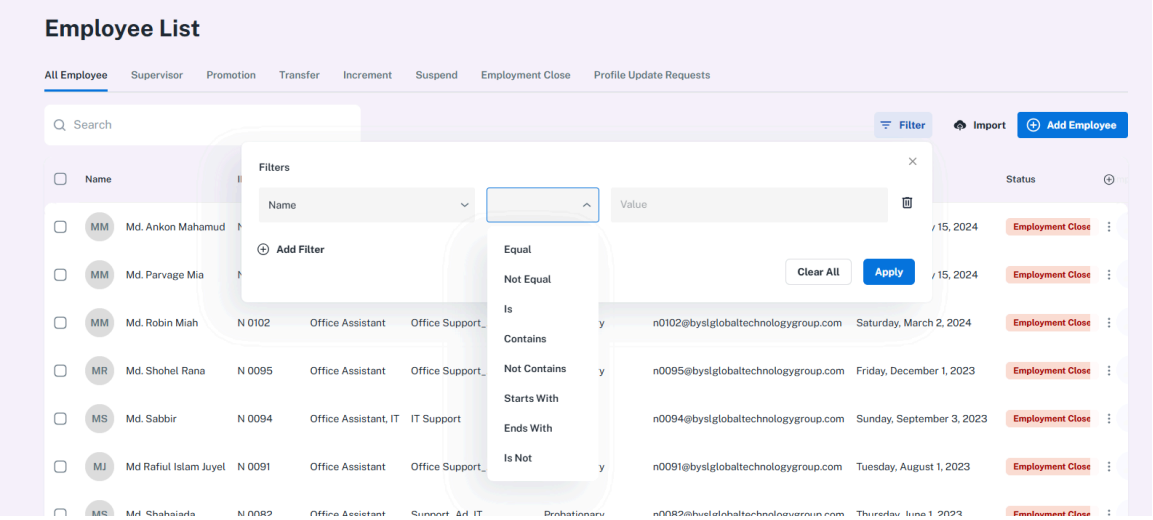


Attribute Selector (Field)

The first dropdown contains **all employee attributes** coming from the system, for example:

- Name
- Employee ID
- Official Email
- Official Phone No
- Personal Email
- Gender
- Religion
- Date of Birth
- Marital Status
- Department
- Designation
- Employment Status
- etc.

These fields are usually mapped directly with **database columns / API keys**



Operator Selector (Condition)

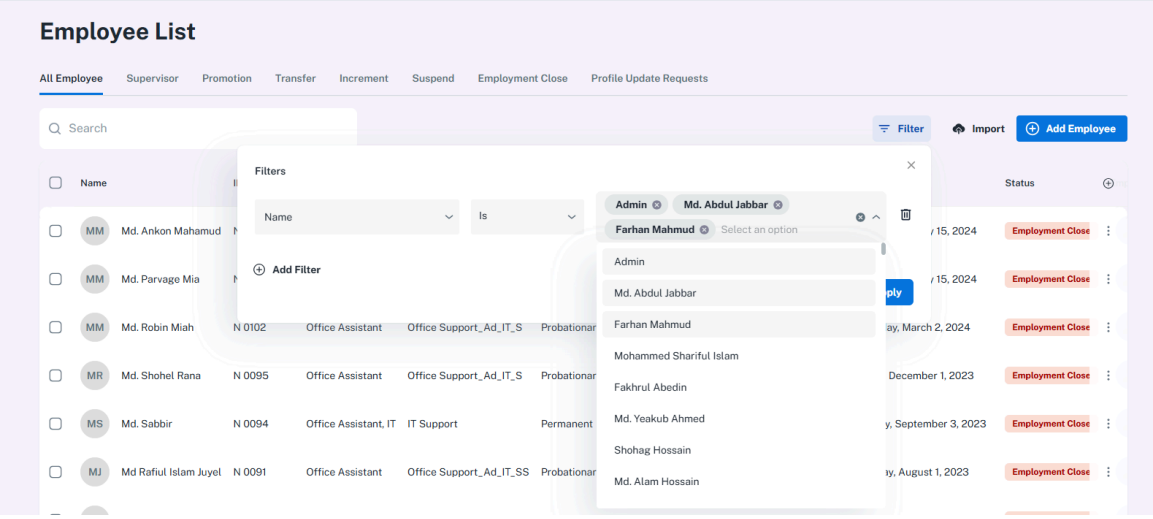
The second dropdown shows operators based on the **selected attribute type**.

Common operators:

- Equal (=)
- Not Equal (!=)
- In
- Not In
- Contains
- Not Contains
- Starts With
- Ends With
- Greater Than (>)
- Less Than (<)
- Greater Than or Equal (>=)
- Less Than or Equal (<=)

Advanced Filter Operators by Field Type:

Field Type	Example	Operators
Text	Name, Email	contains, not contains, =, !=, in, not in
Number	ID, Salary	=, !=, >, <, >=, <=, in
Date	Date of Birth, Joining Date	=, !=, >, <, between
Enum	Gender, Status	=, !=, in, not in



The last field is the **value box**.

Depending on the selected attribute, it can be:

- Free text input
- Date picker
- Number input
- Dropdown with suggestions
- Autocomplete (API-based)

5. Create Origin

All filterable attributes are centrally defined and then **seeded into the database as "origins"**.

These origins are later used by:

- Frontend (to build filter UI dynamically)
- Backend (to validate fields & build secure queries)

Examples of origins:

- Employee List
- Employee ID Card
- Final Settlement
- Promotion
- Transfer
- Increment
- Suspend

Each origin contains:

- Which fields can be filtered
- Which operators are allowed
- What input types will be shown
- Where value options come from

All origins are first defined inside a **central configuration file**:

Suppose in Employee module config.php file:

```
'advance_filter_fields' => [
  AdvanceFilter::ADVANCE_FILTER_TYPE_EMPLOYEE_LIST => [
        'full_name',
        'office_id',
        'official_email',
        'official_phone_no',
```

```
        'personal_email',
        'personal_phone',
    ],
    AdvanceFilter::ADVANCE_FILTER_TYPE_EMPLOYEE_ID_CARD => [...],
    AdvanceFilter::ADVANCE_FILTER_TYPE_EMPLOYEE_FINAL_SETTELEMENT => [...],
    AdvanceFilter::ADVANCE_FILTER_TYPE_EMPLOYEE_PROMOTION => [...],
    AdvanceFilter::ADVANCE_FILTER_TYPE_EMPLOYEE_TRANSFER => [...],
    AdvanceFilter::ADVANCE_FILTER_TYPE_EMPLOYEE_INCREMENT => [...],
    AdvanceFilter::ADVANCE_FILTER_TYPE_EMPLOYEE_SUSPEND => [...],
]
```

Once all origins and fields are defined, they are converted into database records using the seeder:

```
php artisan db:seed --class=AdvanceFilterFieldSeeder
```

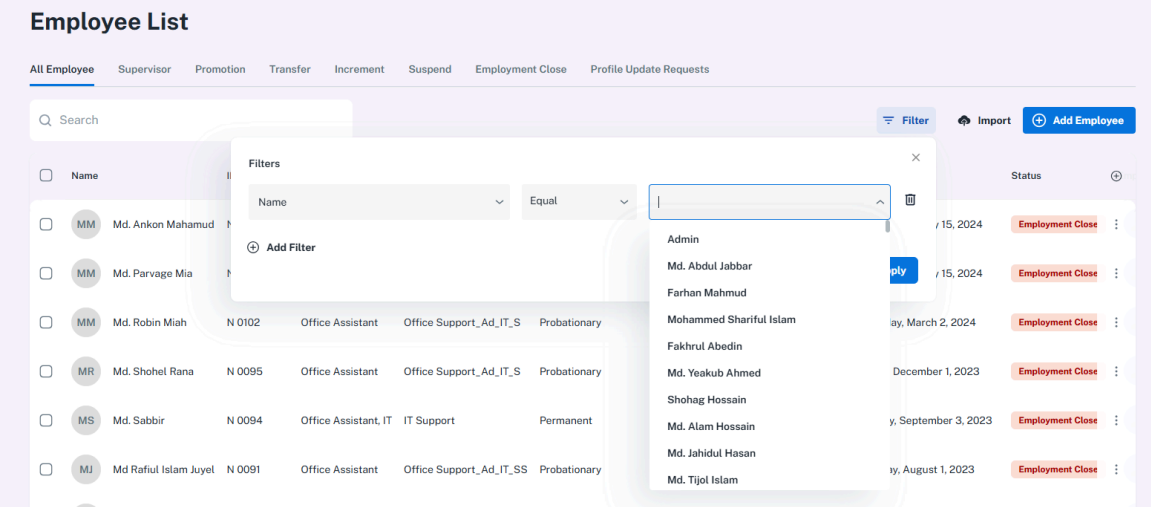
6. Advance Filter Component

1. Overview

The AdvanceFilter component is a reusable React component designed to display dynamic filter forms in the frontend. It allows users to filter table or list data based on multiple criteria including text inputs, select dropdowns, and document types. The component supports default values, custom columns, and optional integration with pagination.

Key Features:

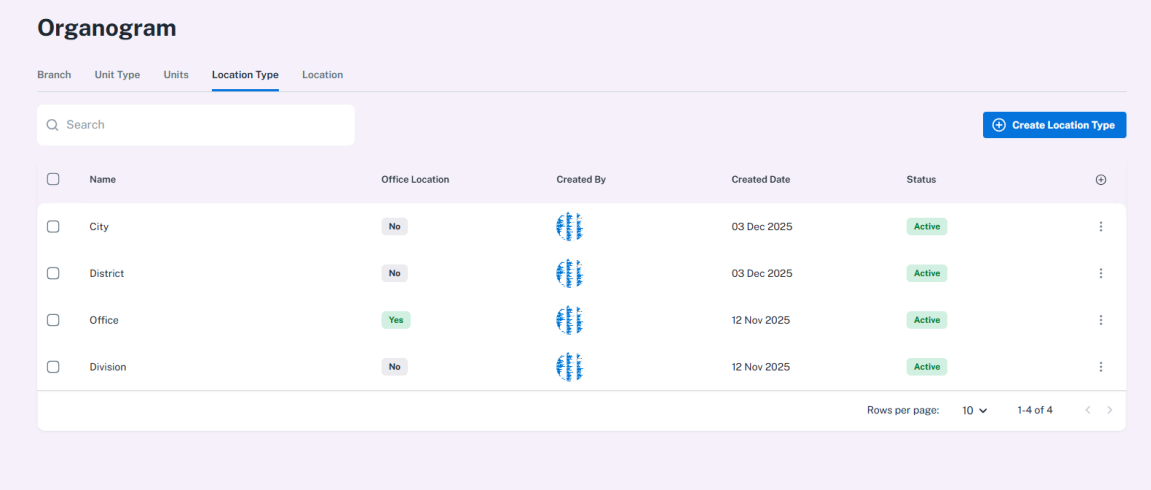
- Dynamically generate filters based on column definitions.
- Supports text and select input types.
- Handles default filters and reset functionality.
- Integrates optional document type dropdowns.
- Works seamlessly with table data fetching via advanceFilterTable function.
- Can be extended for multi-select, date range, and other custom filter types.




```
<AdvanceFilter
        filterColumns={[...filterColumns, ...advanceFilterColumns]}//{filterColumns}
        advanceFilterTable={advanceFilterTable}
        defaultAdvanceFilters={defaultAdvanceFilters}
        initialDataLoadWithAdvanceFilter={initialDataLoadWithAdvanceFilter}
        filterType={advanceFilter}
        customColumns={customColumns}
        docTypeFilter={docType}
        isPagination={isPagination}
      />
```

7. Step By Step Show This Process

1.1 Step One: First create the origin. Suppose you have a module named “Location type” and has a list below like:



Now we have to setup advance filter with this list view for this we have to create origin. Like for “Location Type” now we will be create “location_type” origin. So go to config.php file

```
<?php

use App\Helpers\AdvanceFilter;

return [

  'name' => 'Core',

  'type' => 2,

  'advance_filter_fields' => [

        AdvanceFilter::ADVANCE_FILTER_SETTINGS_LOCATION_TYPE => [

        'name',

        'is_office_location',

        'created_by',

        'created_at',

        'row_status',

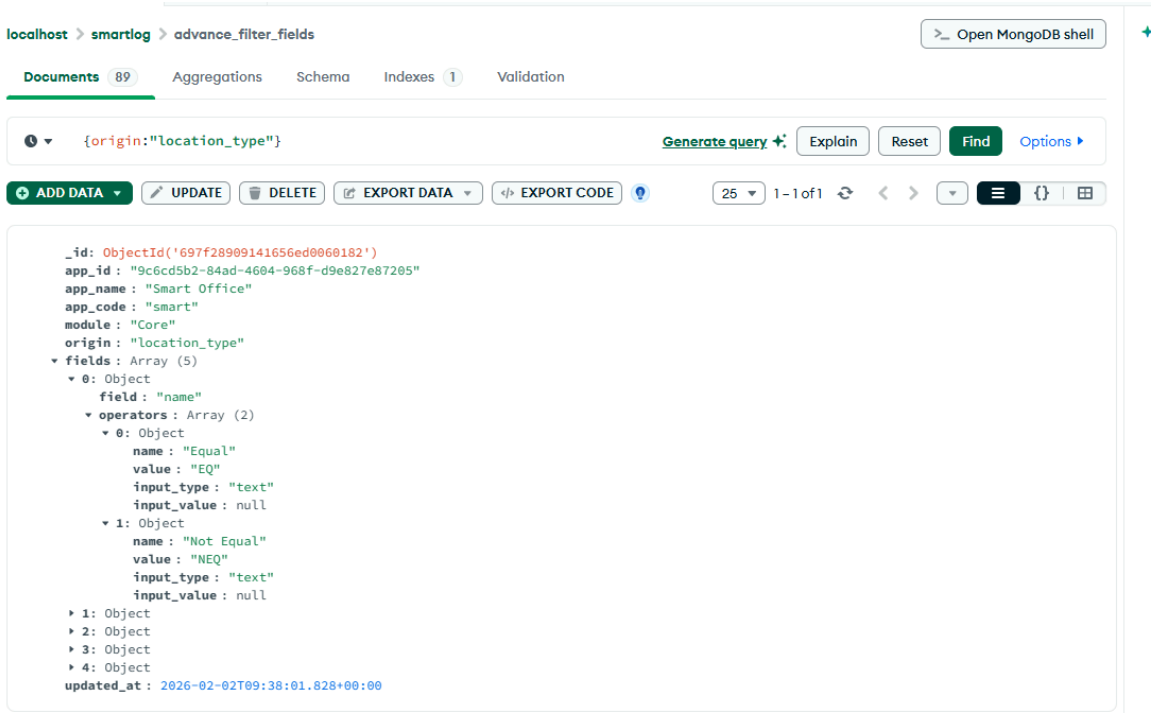
    ],
```

Where in AdvanceFilter.php file we can define bellow this line:

const ADVANCE_FILTER_SETTINGS_LOCATION_TYPE = 'location_type';

1.2 Step Two: In this we have run command for initialize origin and insert into MongoDB as a new document. Run: php artisan db:seed --class=AdvanceFilterFieldSeeder

Here the document data pic of MongoDB after command run



1.3 Step Three: Now we can setup this origin with frontend view as a advance filter form. For that we have to pass “advanceFilter={"location_type"}”

```
<LocationTypesDataTable
  data={data?.data ?? {}}
  defaultColumns={[...(filter?.fields?.split(",") || [])]}
  handleAction={handleAction}
  advanceFilter={"location_type"}
  isLoading={isLoading}
  actionButton={actionButton}
  downloadMenuItems={EXPORT_TYPE_EXCEL}
/>
```

Your app likely has **multiple tables**, for example:

- Location Types
- Employees
- Attendance
- Departments
- Companies

Each of these:

- Has **different filterable fields**
- Uses **different APIs**
- Needs **separate saved filters / default filters**

So advanceFilter works as a **context key**.

advanceFilter="employees"

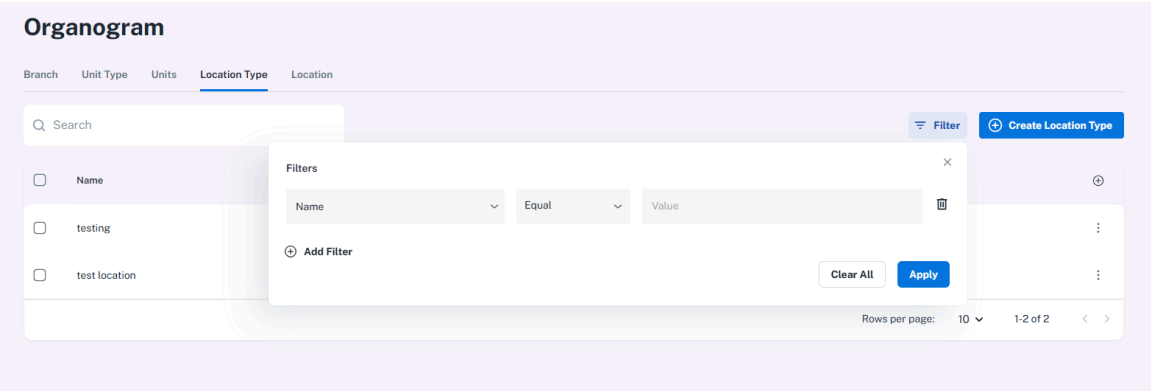
advanceFilter="location-types"

advanceFilter="attendance-report"

Internally AdvanceFilter component, it's used like:

api/smart/advancefilter/utility/advance-filter-fields/null/location_type

After AdvanceFilter component call show the filter form like:



1.4 Step Four: Now when we submit this form for advance filter by apply button click in backend code process this filter parameters by this query like:

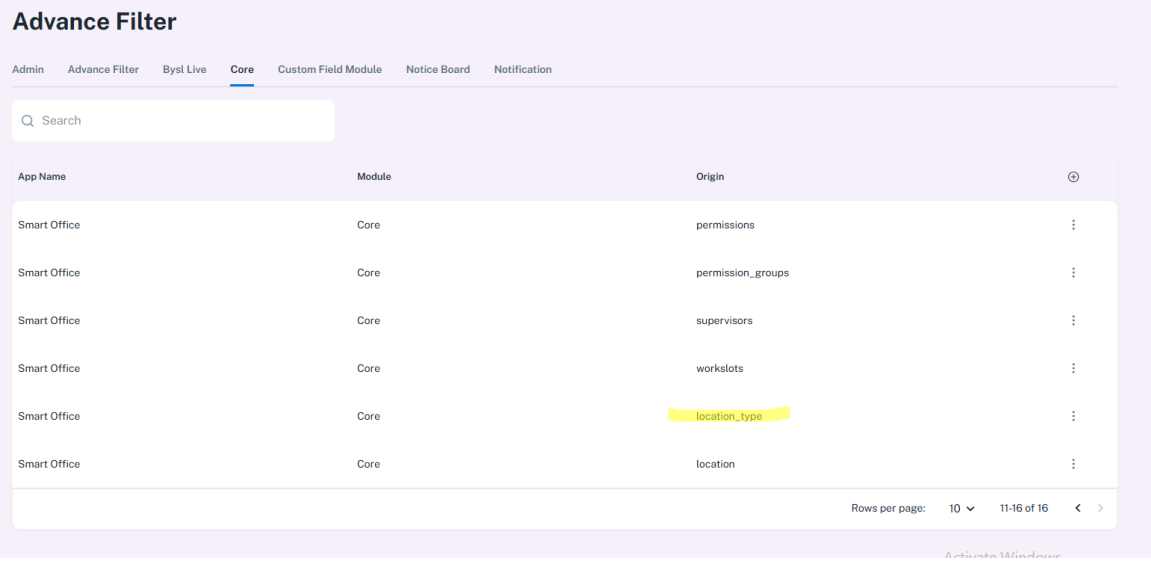
```
if (request()->has('advance_filter')) {

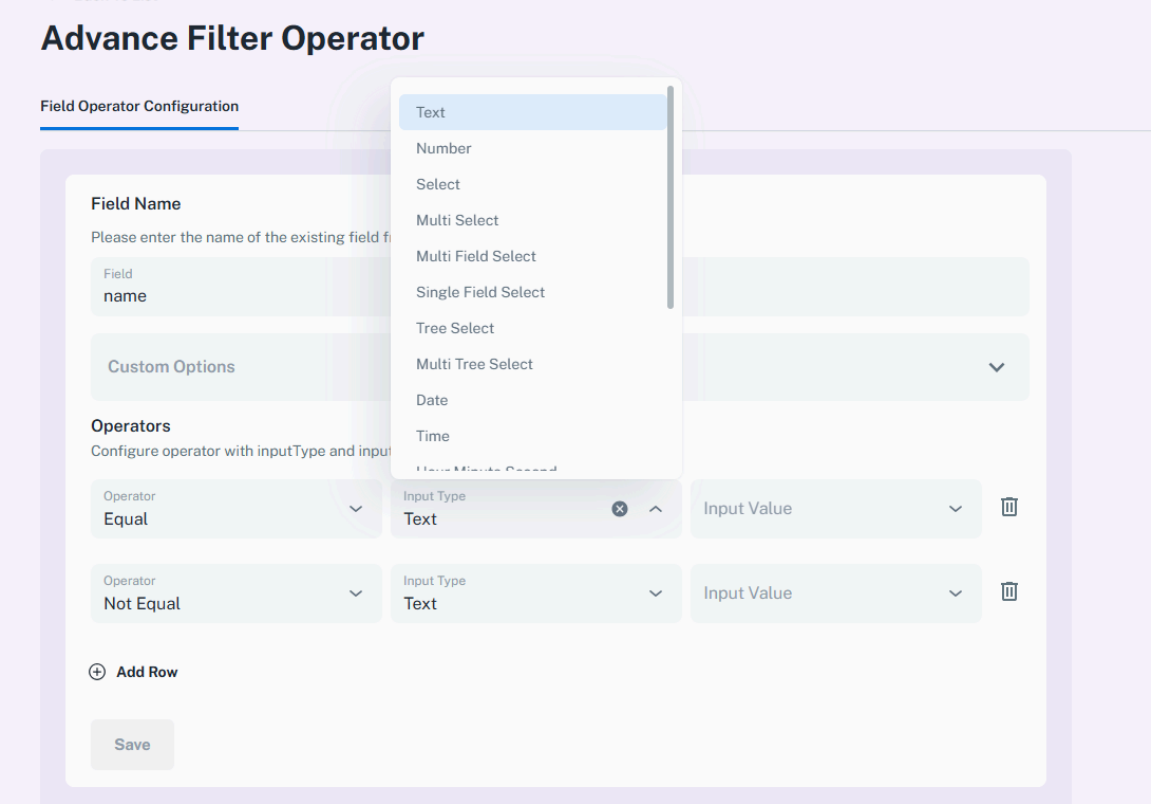
$advanceFilterQuery =
AdvanceFilter::decodeAndDecompressData(\request()->get('advance_filter'));

$hasAttributes = array_filter($advanceFilterQuery, fn($filter) =>in_array($filter['attribute'],
[
            'name',
            'is_office_location',
            'created_at',
            'created_by',
            'row_status'
 ]));
        foreach ($advanceFilterQuery as &$item) {
            $attribute = $item['attribute'];
            $date = $item['value'];
            if ($attribute === 'created_at') {
                $item['attribute'] = 'date_time';
                $item['value'] = ['created_at',$date,''];
            }
        }
        if ($hasAttributes) {
    HelperService::customAdvanceFilterNestedRelation($queryBuilder, dvanceFilterQuery);
        }
      }
```

We use customAdvanceFilterNestedRelation() function for advance filter query build.

1.5 Step Five: We can setting origin filter with operators by login with system admin panel like:





.env settings with mongoDB like:

```
#mongodb connection

MONGO_DB_HOST=127.0.0.1
```

```
MONGO_DB_PORT=27017

MONGO_DB_DATABASE=smartlog

MONGO_DB_USERNAME=

MONGO_DB_PASSWORD=
```